

SDK / стек

Название приложения	888Casino
Android Gradle Plugin	8.7.3
minSdk	26
targetSdk	36
Kotlin	да, 2.0.21
Web View	да
Custom Tabs	нет
Рекламные сети	нет
Аналитика	нет
Permissions	android.permission.INTERNET, android.permission.ACCESS_WIFI_STATE, android.permission.ACCESS_NETWORK_STATE, com.luckyreads.bookshelf.DYNAMIC_RECEIVER_NOT_EXPORTED_PERMISSION, com.android.vending.CHECK_LICENSE
Libraries	Kotlin 2.0.21, kotlin-coroutines 1.7.3, OkHttp 4.12.0, AndroidX AppCompat 1.7.0, AndroidX Activity 1.9.3, AndroidX Core/Core-KTX 1.13.1, AndroidX ConstraintLayout, AndroidX Lifecycle 2.8.7, AndroidX Room 2.6.1, AndroidX RecyclerView 1.3.1, AndroidX Fragment, AndroidX Emoji2 1.3.0, AndroidX Startup, AndroidX ProfileInstaller, AndroidX CardView, AndroidX CoordinatorLayout, AndroidX ViewPager2, AndroidX Transition, AndroidX SavedState, AndroidX SQLite, Material Components 1.12.0, Pairip licensecheck (com.pairip.application / LicenseActivity)
Подозрительные домены	blue-limit-83a7.serg-savwork.workers.dev
SharedPreferences	файл cfg: ключ rurl — сохранённый финальный URL после редиректа cloaking-сервера; при следующих запусках открывается сразу без повторного запроса
Есть ли клоака	да
Подозрительные слова	casino, WebView, User-Agent, rurl, redirect, workers.dev, license check, white page (assets/index.html)

Проверка домена: blue-limit-83a7.serg-savwork.workers.dev

Домен	blue-limit-83a7.serg-savwork.workers.dev
Детекции	нет
Куда редиректит	нет
Что выводит (кратко)	HTTP 200, content-type application/json, тело: {"status":"ok","version":"1.0.0","ts":...}; cache-control: no-store. С текущей (датацентр) сети редиректа нет — решение «белый/боевой» на стороне Worker (динамический оффер-URL при успешной проверке).
Где припаркован	Cloudflare Workers (поддомен *.workers.dev / serg-savwork)

Как работает клоака В начале

Снаружи в Google Play приложение называется «888Casino», а внутри пакета и в интерфейсе это «Lucky Reads» — каталог из 100 классических книг с кнопками Browse / Lucky Pick / Search. Обман в том, что при старте приложение тихо спрашивает у своего Cloudflare Worker, можно ли этому человеку показывать «книжную» оболочку или нужно отправить его дальше. Модератору и «неподходящему» трафику показывают локальную белую страницу с книгами. «Нужному» пользователю сервер отвечает редиректом на другой адрес — приложение сохраняет этот адрес и открывает его во внешнем браузере. Финальный сайт в коде не зашит: он приезжает динамически с cloaking-сервера.

2) Кому что показывают

Белая страница / обычный режим. Её видит тот, для кого запрос к Worker не приводит к смене URL (нет HTTP-редиректа) либо сеть упала с ошибкой. В WebView загружается `file:///android_asset/index.html` — полноценный офлайн-каталог «Lucky Reads» (поиск, список, случайная книга). Так выглядит безопасный контент для проверки магазина.

Чёрный / боевой режим. Тот, кого сервер «пропустил» (OkHttp видит, что итоговый URL ответа отличается от исходного запроса), получает сохранённый `redirect-URL`. WebView прячется, и система открывает этот URL через `Intent.ACTION_VIEW` во внешнем браузере. При следующих запусках кэш `rurl` сразу ведёт туда же, без повторного опроса.

3) Как приложение решает, кого куда пустить

- Свой API (Cloudflare Worker) → смотрит запрос `https://blue-limit-83a7.serg-savwork.workers.dev/?app=com.luckyreads.bookshelf` → если URL после ответа тот же — белая страница; если другой (редирект) — боевой URL.
- User-Agent → в заголовок кладётся `WebSettings.getDefaultUserAgent()` (как у системного WebView) → сервер может отличать реальное Android-устройство от бота/скрипта.
- Модель устройства → заголовок `X-Device-Model = Build.MODEL` → дополнительный сигнал для серверной фильтрации.
- Язык (Accept-Language) → жёстко `en-US,en;q=0.9` → сервер видит «английский» клиент; гео/страна в клиентском коде отдельно не читаются (в коде не видно `Locale/TelephonyManager`) — гео, если есть, скорее на стороне Worker по IP.
- `SharedPreferences` `cfg` / `rurl` → если боевой URL уже сохранён → повторный запрос к серверу не делается, сразу открывается кэш.
- Ошибка сети → при сбое OkHttp вызывается белый режим (локальный `index.html`) — безопасный fallback.

4) Цепочка по шагам

Шаг 1. Пользователь открывает приложение → стартует `MainActivity` → готовится скрытый `WebView` и читаются `SharedPreferences` `cfg`.

Шаг 2. Если в ключе `rurl` уже есть непустая ссылка → приложение считает пользователя «уже проверенным» → сразу открывает её во внешнем браузере (боевой режим без нового запроса).

Шаг 3. Если `rurl` пуст → собирается URL Worker с параметром `?app=имя_пакета` → зачем: сервер знает, какое именно приложение стучится.

Шаг 4. OkHttp отправляет GET с заголовками `X-Device-Model`, `Accept-Language` и `User-Agent` → сервер получает сигналы устройства и клиента → решает, редиректить или ответить «на месте».

Шаг 5. При ответе сравниваются исходный URL запроса и итоговый URL ответа (поле `HttpUrl` после следования редиректам) → это и есть клиентская развилка белый/чёрный.

Шаг 6. URL совпал (редиректа не было) или сеть упала → загружается `file:///android_asset/index.html` в `WebView` → пользователь видит каталог классики Lucky Reads.

Шаг 7. URL изменился → итоговая ссылка пишется в `cfg/rurl` → запоминается для следующих запусков.

Шаг 8. `WebView` скрывается, стартует `Intent.ACTION_VIEW` на этот URL → человек уходит во внешний браузер на динамический лендинг/оффер.

Шаг 9. Если браузер не открылся (исключение) → fallback снова на белую страницу в `WebView` → приложение не «ломается» на виду.

5) Технические уловки

- XOR-шифрование строк (метод `MainActivity.t`, ключ `[193, 30, 135, 90]`) → прячет домен Worker, имена `prefs` (`cfg/rurl`), заголовки, путь к `assets` → чтобы простой `grep` по APK не находил cloaking-URL (именно поэтому `domain_checks` пайплайна сначала «не увидел» домен).
- Решение «по факту редиректа», а не по JSON → тело `status/ok` при белом ответе клиент даже не разбирает → логика проще и меньше следов «if offer then...» в коде.
- Боевой URL во внешнем браузере, белый — в `WebView` → модератор видит «приложение-читалку»; целевой трафик уходит из `WebView` на сторонний сайт.
- `usesCleartextTraffic=true` и `Mixed Content Mode = 0` → разрешены незащищённые/смешанные загрузки на случай http-офферов.
- `Pairip licensecheck` — обёртка Google Play Integrity/лицензии, не часть клоаки контента, но добавляет «магазинный» вид приложения.

6) Роль подозрительных доменов

blue-limit-83a7.serg-savwork.workers.dev — появляется на шагах 3–5: это «мозг» решения. С него не качается игра; он либо оставляет запрос на том же адресе (белый JSON ok), либо редиректит на динамический оффер-URL. Хост на Cloudflare Workers (serg-savwork). В готовом domain_checks.json пайплайна домен отсутствовал из-за XOR в DEX; восстановлен расшифровкой строк MainActivity. VirusTotal-цифры не выдумывались.

7) Финал для пользователя

После всех проверок «боевой» пользователь попадает на динамический URL, который присылает Worker через HTTP-редирект. В самом APK конечный сайт (казино/лендинг/оффер) не прописан — его нельзя прочитать статически. Название приложения в сторе «888Casino» при белой маске «Lucky Reads» указывает на гемблинг-ориентированный оффер, но конкретный land с текущей сети не раскрылся (Worker ответил 200 JSON без Location). Повторные запуски после первого «прохода» сразу открывают закешированный url во внешнем браузере.